Real-Time Messaging

gateways hold sockets, the backplane crosses servers

Watch the 4-min video

In one line: clients keep WebSocket connections to a fleet of thin **gateways**; a chat service **stores** each message first, looks up the recipient's gateway in a **session registry**, and publishes to that gateway over a **pub/sub backplane**. Sequence numbers, acks and sync-on-reconnect make delivery reliable even when the recipient is offline.

1 Two servers, one chat

Asha is connected to WebSocket server 3; Ben is connected to server 17. She hits send. Server 3 has her message but no socket to Ben. With millions of open connections spread over hundreds of servers, the design must answer three questions: **where is Ben?**, **how does the message cross servers?** and **what if Ben is not there at all?** The target is a delivery time of tens of milliseconds, with no lost or doubled messages.

2 Picking the transport

Transport	Direction	How it works	Use for
WebSocket (RFC 6455)	both ways	HTTP/1.1 `Upgrade` → `101 Switching Protocols`, then framed messages on one TCP (TLS) connection; ping/pong control frames	chat, games, collaboration
Server-Sent Events	server → client	one long HTTP response of `text/event-stream`; browser auto-reconnects with `Last-Event-ID`	live feeds, scores, notifications
Long polling	server → client	request is held open until data arrives, then the client asks again	fallback when sockets are blocked
Mobile push (APNs/FCM)	server → device	OS-level channel that wakes a sleeping app; best effort	offline recipients only (lesson 28)

Chat needs both directions and low overhead per message, so WebSocket is the default. A message frame adds only 2–14 bytes of header, compared with hundreds of bytes of HTTP headers per poll.

3 The building blocks

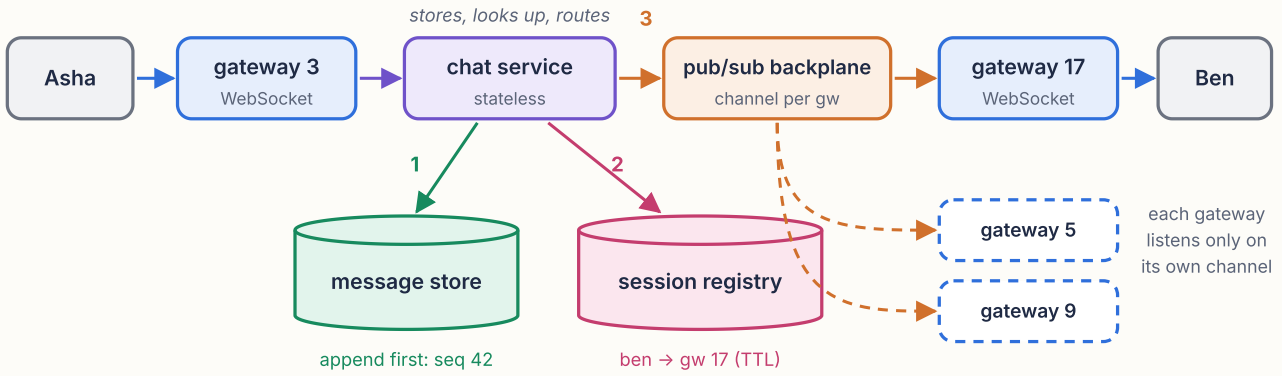


Figure 1. Asha's message: the chat service (1) appends it to the store, (2) asks the registry where Ben is, (3) publishes on gateway 17's channel. Gateways never call each other.

Connection gateways. A gateway terminates the WebSocket, authenticates it once (token at connect time), and moves frames in and out. It holds no business logic, so it is cheap to scale out. Each open connection costs a file descriptor and some kilobytes of buffers, so a tuned server holds hundreds of thousands of connections (WhatsApp reported over 2 million per Erlang server in 2012). Gateways sit behind a layer-4 load balancer; on deploy they are **drained** slowly, and clients reconnect with **jittered exponential backoff** so a restart does not cause a reconnect storm.

Session registry. When Ben connects, the gateway writes `session:ben → {gw-17, conn-881}` into a fast store such as Redis. One user usually has several sessions (phone, laptop, tablet), so store a set or hash per user, not one value. Each entry has a TTL that heartbeats refresh (lesson 27): if gateway 17 crashes without cleaning up, its entries expire on their own.

Backplane. Every gateway subscribes to one channel named after itself (`gw-17`). The chat service publishes to the channels of the recipient's gateways. Options:

- **Redis Pub/Sub:** very fast, but fire-and-forget: a message published while nobody is subscribed is simply gone. That is fine *only* because the store already holds it and the client will sync.
- **Kafka** (or another durable log): replayable and ordered per partition, at the cost of a few more milliseconds.
- **Direct RPC** from chat service to gateway: simplest at small scale, but every service must know every gateway's address.

Large systems often route by *conversation* instead of by user: Slack's published design maps each channel to a **channel server** by consistent hashing, and gateways subscribe to the channels their users have open. Either way, the rule is the same: gateways only listen; they never talk to each other.

4 Message IDs, order and acknowledgements

- **Client message ID** (e.g. a UUID, `m-7f3`) is created on Asha's phone before sending. If the network drops and she retries, the server sees the same ID and does not store a second copy: the send is **idempotent**.
- **Sequence number** is assigned by the server, per conversation (42, 43, 44 ...). It defines one order everyone agrees on, and it lets a client detect gaps. Give each conversation a single writer: e.g. partition by conversation ID so one partition (or one row with an atomic counter) owns the counter. Time-sortable IDs such as Twitter/Discord Snowflakes (timestamp + worker + counter) are another option.

- **Three ticks:** *sent* = stored by the server; *delivered* = Ben's device received it; *read* = displayed on screen. Receipts travel back over the same gateways and backplane. Send a **read watermark** ("read up to 57") instead of one receipt per message: one write covers many messages.

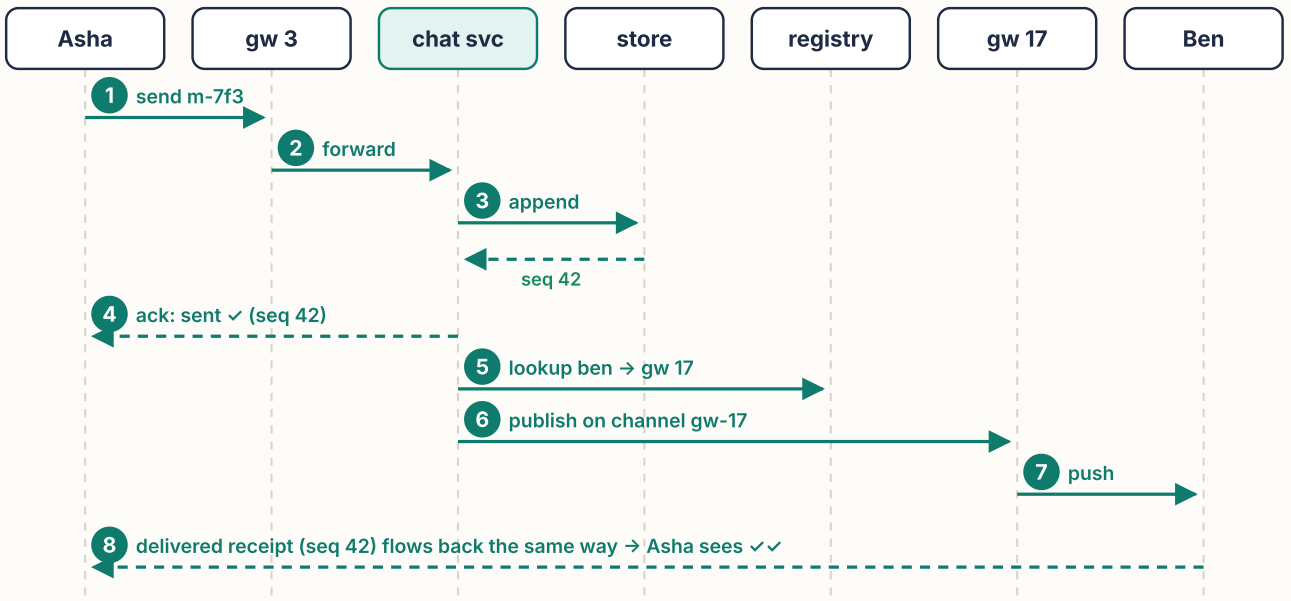


Figure 2. End to end. The message is stored (3) before anything is delivered, so a crash after step 3 loses nothing. The **sent** tick comes from the server; **delivered** and **read** come from Ben's device.

Delivery is **at-least-once** (retries can repeat a push) plus **deduplication** on the client by message ID, which is what users experience as exactly-once. Ordering is guaranteed per conversation only; across conversations nobody needs it.

5 Offline users, push and sync



Figure 3. Sync by sequence number. Because the log is the source of truth, a missed push or dropped socket only delays messages; it never loses them.

If the registry has no session for Ben, the message is already safe in the store. The chat service asks the notification system (lesson 28) to send a push through APNs or FCM, which wakes his phone. Push is best effort and may be collapsed by the OS, so it carries at most a preview and is never the delivery mechanism itself. On reconnect, the app sends its last seen sequence number per conversation (or a single per-user sync cursor across all conversations) and downloads the gap.

6 Groups and storage

Groups: the chat service stores the message once, then looks up every member's sessions and publishes once per *gateway*, not once per member: 300 members spread over 40 gateways means 40 publishes, and each gateway pushes to its local sockets. Very large channels switch to subscription:

gateways subscribe to the channel topic, and members fetch history when they open it. Receipts in big groups are usually aggregated or turned off.

Message store: chat is append-heavy and read by "latest N in this conversation", which fits wide-column stores. Partition by `(conversation_id, time_bucket)` and cluster by message ID, so one conversation's recent messages sit together and a huge, old conversation does not grow one partition forever. Discord stored messages this way in Cassandra and later moved to ScyllaDB.

WORKED EXAMPLE: GATEWAY 3 → GATEWAY 17

Asha sends "hello" with client ID `m-7f3` on gateway 3. The chat service appends it to conversation asha–ben as **seq 42** and acks Asha (one tick). It reads `session:ben` → gateway 17 and publishes the message on channel `gw-17`. Gateway 17 pushes it down Ben's socket; his app stores it, sends *delivered (42)*, and Asha sees two ticks. Each hop is about a millisecond inside one data centre, so the round trip is tens of milliseconds. If Ben's phone had been asleep, the lookup would find no session, a push would go out, and his app would fetch `after=41` on reconnect.

TRADE-OFFS

- Redis Pub/Sub is fastest but loses messages with no subscriber; Kafka is durable but adds latency and operational weight.
- Per-user routing is simple for 1:1 chat; per-channel routing scales better for big groups.
- Short heartbeats detect dead sockets fast but cost battery and traffic on millions of devices.
- Storing before delivering adds a write to the latency path, but it is what makes offline sync and retries safe.

COMMON MISTAKES

- Gateways calling each other directly: an N×N mesh that breaks every time a gateway is added or removed.
- Treating the backplane or a push notification as the source of truth instead of the message store.
- Ordering by client timestamps: phone clocks are wrong; use server-assigned sequence numbers.
- One registry value per user, so logging in on a laptop silently stops delivery to the phone.

7 Key terms

Gateway: server that holds client WebSocket connections and nothing else

Backplane: pub/sub layer that carries messages between servers

Client message ID: ID made on the sender's device, used to dedupe retries

Session registry: user → gateway(s) map with TTL, often in Redis

Sequence number: server-assigned order within one conversation

Sync cursor: last sequence a device has; it fetches everything after it

8 Interview questions

Q1. How do two users on different WebSocket servers exchange messages? BASIC

The sender's gateway hands the message to a chat service, which stores it, finds the recipient's gateway in the session registry, and publishes it over pub/sub to that gateway, which pushes it down the socket.

Q2. What if the recipient is offline? BASIC

It is already stored, so nothing is lost: send a push notification, and on reconnect the client syncs everything after its last sequence number.

Q3. Why WebSocket rather than HTTP polling for chat? BASIC

It is a persistent, bidirectional connection with tiny per-message framing, so the server can push instantly without the latency and header overhead of repeated requests.

Q4. A user is logged in on a phone and a laptop. How does routing change? SCENARIO

The registry stores a set of sessions per user; the service publishes to every gateway that holds one, and each device acks and syncs on its own cursor.

Q5. Gateway 17 crashes with 300,000 connections. What happens? SCENARIO

Clients reconnect with jittered backoff to other gateways, which write new registry entries; old entries expire by TTL; each client syncs from its cursor, so no messages are lost.

Q6. How do you send a message to a 5,000-member group efficiently? SCENARIO

Store it once, then publish once per gateway that has members (or to a channel topic the gateways subscribe to); turn off or aggregate per-member receipts.

Q7. How do you guarantee the same message is never shown twice? TRICKY

You can't avoid duplicate delivery under retries, so make it harmless: a client-generated message ID makes the server write idempotent, and the client dedupes by ID.

Q8. Redis Pub/Sub drops messages when no one is subscribed. Why is it still acceptable here? TRICKY

Because the message store, not the backplane, is the source of truth: a dropped publish only delays the message until the client's next sync.

Q9. How do you keep message order correct? TRICKY

Assign a sequence number per conversation from a single writer (partition by conversation ID); clients sort by it and fetch any gaps, never trusting device clocks.

Next: SD-Q-002 · Presence backendroadmap.com